

A Pragmatic Guide to Building Conservative Discrete Abstractions of Cyber-Physical Systems

Jordan Peper, Krish Kapadia, James Gast, Ethan Howes, and Ivan Ruchkin

University of Florida, Gainesville FL 32611, USA

Abstract. Symbolic model checking is an effective approach for verifying semantically rich temporal-logic properties of cyber-physical systems, but it hinges on discretizing continuous-state dynamics into a finite-state abstraction. To transfer verification guarantees from the abstract model to the concrete CPS, the abstraction must conservatively approximate the concrete state space and behaviors. Hence, model-builders must maintain this soundness while balancing pessimism with tractability. However, they face several common pitfalls such as under-covering the state space, under-approximating transitions, unsound pruning of “degenerate” behaviors, and improper specification lifting. This tutorial presents a pragmatic, conservative-by-construction workflow for building discrete abstractions of closed-loop dynamical systems. The workflow consists of four modular steps with interchangeable subroutines: (i) state-space tessellation and abstraction-function design, (ii) conservative transition construction via axis-aligned bounding boxes, polytopes, or sampling with PAC coverage certificates, (iii) mitigation of spurious transitions and self-loops using certified erasure and counterexample-guided abstraction refinement, and (iv) sound lifting of LTL specifications using may/must semantics. We demonstrate the end-to-end pipeline on three case studies and report how these design choices affect abstraction structure, runtime, and verification outcomes.

Keywords: dynamical systems, discrete abstraction, model checking

Code: github.com/Trustworthy-Engineered-Autonomy-Lab

Desired length: Half-day

1 Introduction

Cyber-physical systems (CPS) operate in environments where failures can threaten life, property, and the environment. Accordingly, there is sustained demand for rigorous correctness assurance for realistic CPS. Stemming from a long history of verification in hardware and software, symbolic model checking remains particularly effective for verifying CPS when requirements are expressed as semantically rich temporal-logic specifications [4, 15]. Model checking performs an exhaustive, specification-driven exploration of a model’s state space to determine whether a given safety property holds [4, 7].

To be computationally tractable, model checking requires a *finite-state* model, whereas real-world CPS evolve over continuous state spaces. Hence, engineers

cast the CPS as a finite state model (called an *abstraction* of the CPS) by aggregating the states into a finite set of abstract states, and then build transition relations over them. However, for guarantees to transfer from this abstraction to the concrete CPS, the abstraction must conservatively approximate the state space and behaviors of the concrete CPS.

In practice, creating a useful conservative abstraction requires carefully maintaining soundness and balancing pessimism against computational cost. Inexperienced model-builders tend to make four typical errors: (i) under-covering the concrete state space with the one, (ii) under-approximating the abstract transition relation (thereby omitting feasible behaviors), (iii) pruning degenerate behaviors without a certificate that preserves soundness, and (iv) lifting the specification without preserving the implication from abstract to concrete satisfaction.

This tutorial presents a single, sound *model-building workflow* with standard techniques to avoid the aforementioned pitfalls. The workflow consists of four steps (Sec. 5–8) with interchangeable subroutines and results in a conservative-by-construction, sound abstraction. First, we explain how to construct an abstract state space using a rectilinear tessellation. Second, we present three approaches for building a conservative transition relation: (a) an axis-aligned bounding box over-approximation, (b) a less conservative polytope technique, and (c) a sampling-based construction equipped with a PAC-style certificate. Third, we describe common degenerate behaviors in conservative abstractions (including spurious transitions and self-loops) and mitigation strategies: certified self-loop erasure (with both a deterministic and probabilistic guarantee) and counterexample-guided abstraction refinement (CEGAR). Fourth and finally, we address the specification-lifting step by translating the concrete LTL formula to an abstract formula over the abstract state space to ensure that its satisfaction on the abstraction is sufficient to satisfaction on the concrete CPS.

The rest of this paper is organized as follows. Sec 2 revisits verification literature for cyber-physical systems. Sec. 3 introduces relevant preliminaries used throughout the tutorial, including conservative abstraction, LTL lifting, and model checking. Sec. 4 provides an in-depth look at the four abstraction steps discussed in detail in Sec. 5– 8. Finally, Sec. 9 demonstrate the full abstraction pipeline on three case studies, and then concludes with Sec. 10.

2 Literature Landscape

Verification of dynamical systems. Verification of complex dynamical systems, including hybrid systems [24], is commonly organized into three methodological families: (i) deductive methods, (ii) explicit-state reachability analysis, and (iii) symbolic abstraction-based techniques [1, 17, 23]. Deductive methods exploit analytical system properties (e.g., invariants, Lyapunov arguments, or proof rules) to certify stability or safety [17]. Explicit-state reachability analysis propagates an initial set forward through the dynamics to compute an over-approximation of the reachable state tube, enabling detection of safety violations [11]. Finally, symbolic abstraction-based techniques construct a simpler

finite-state model that soundly over-approximates the behaviors of the concrete system, thereby enabling the use of symbolic model checking [17]. This tutorial concentrates solely on abstraction-based techniques.

Abstraction of dynamical systems. Casting a dynamical system as a symbolic abstraction requires reducing the state and action spaces to a finite-state over-approximation that preserves ground-truth behaviors. Typical abstraction schemes include phase-based [20, 21, 29], where rectangular or linear flow constraints are placed on the system states and dynamics, and predicate-based [2, 3, 27], where continuous state spaces are abstracted as a *tessellating* collection of predicate-defined subspaces. This tutorial focuses on the latter.

Counterexample-guided abstraction refinement (CEGAR). Methodologies for refining transition systems that mitigate the state explosion problem in symbolic model checking are rooted in early software verification. The early 2000s saw the development of automated refinement loops that iteratively rewrite abstraction predicates until spurious behaviors, manifested as erroneous abstract transitions, are eliminated [25]. A canonical representative is counterexample-guided abstraction refinement [14]. Although these methods were initially developed for purely discrete-state systems, they were later extended to timed automata [16, 29] and hybrid systems [2, 12, 13, 19]. Subsequent work applied CEGAR to probabilistic abstractions, including Markov models [9, 10, 26]. This tutorial revisits CEGAR in the context of refining discrete abstractions.

3 Background

We describe the preliminaries to conservative discrete-state abstraction, linear temporal logic (LTL) specification, and model checking.

Definition 1 (Discrete-time dynamical system). A discrete-time, continuous-state dynamical system is a tuple $s = (X, X_0, f)$ where:

- $X \subseteq \mathbb{R}^n$ is the (continuous) state space
- $X_0 \subseteq X$ is the set of initial states
- $f : X \rightarrow X$ is the closed-loop discrete-time dynamical model

The dynamical system acts in discrete time, i.e., $x_{k+1} = f(x_k)$, where $k \in \mathbb{N}$ is a time index. We assume that f is a known deterministic function. Throughout this paper, we let $f(R) = \{f(x) \mid x \in R\}$ denote the *image* of $R \subset X$ under f . Furthermore, we denote the set of all trajectories τ of the system as $\llbracket s \rrbracket$:

$$\llbracket s \rrbracket = \bigcup_{x_0 \in X_0} \tau(s, x_0) \quad (1)$$

$$\tau(s, x_0) = \left\{ x : \mathbb{N} \rightarrow X \mid \forall k \in \mathbb{N}, x_{k+1} = f(x_k) \right\} \quad (2)$$

We aim to cast this dynamical system as a *finite state machine*:

Definition 2 (Finite state machine). A finite state machine (FSM) is a transition system $\hat{s} = (\hat{X}, \hat{X}_0, \Sigma, F)$ where:

- $\hat{X}$ is a finite collection of states
- $\hat{X}_0 \subseteq \hat{X}$ is the set of initial states
- Σ is an alphabet of action labels
- $F \subseteq \hat{X} \times \Sigma \times \hat{X}$ is the state transition relation

The nondeterminism in F is resolved by a memoryless policy $\pi : \hat{X} \rightarrow \Sigma$ drawn from a set of admissible policies Π . Once a particular $\pi \in \Pi$ is fixed, the abstract system behaves deterministically, inducing a map between abstract states that we denote by $F_\pi : \hat{X} \rightarrow \hat{X}$. We define the (single) abstract trajectory that starts at $\hat{x}_0$ when nondeterminism is resolved by π :

$$\tau_\pi(\hat{s}, \hat{x}_0) = \{x : \mathbb{N} \rightarrow X \mid \forall k \in \mathbb{N}, \hat{x}_{k+1} = F_\pi(x_k)\}. \quad (3)$$

To capture the full behavior of the abstraction, we then collect *all* trajectories that can originate from $\hat{x}_0$ under any scheduler:

$$\tau(\hat{s}, \hat{x}_0) = \left\{ \hat{x} : \mathbb{N} \rightarrow \hat{X} \mid \forall k \in \mathbb{N}, \exists \sigma_k : (\hat{x}_k, \sigma_k, \hat{x}_{k+1}) \in F \right\} = \bigcup_{\pi \in \Pi} \tau_\pi(\hat{s}, \hat{x}_0). \quad (4)$$

Finally, allowing the initial condition to vary over $\hat{X}_0$ gives the overall set of plausible abstract trajectories:

$$\llbracket \hat{s} \rrbracket = \bigcup_{\hat{x}_0 \in \hat{X}_0} \tau(\hat{s}, \hat{x}_0). \quad (5)$$

Our objective is to synthesize such an FSM that conservatively approximates the behavior of our concrete dynamical system. We will fit our system into this template first by discretizing the state space X , i.e., partitioning it into smaller subspaces through an *abstraction function* $\psi : X \rightarrow \hat{X}$, such that $\psi(x) = \hat{x}$. The inverse of this abstraction function (*concretization*, $\psi^{-1} : \hat{X} \rightarrow 2^X$) on an abstract state $\hat{x}$ returns the concrete subset that the state represents:

$$\psi^{-1}(\hat{x}) = \{x \mid \psi(x) = \hat{x}\} \subset X \quad (6)$$

Next, we must determine the feasible transition relation F between abstract states according to the concrete system dynamics f . This tutorial will cover various techniques to build this set of transition relations. However, we can only guarantee that our FSM over-approximates our concrete system if it is a *conservative system abstraction*:

Definition 3 (Conservative system abstraction). A finite state machine $\hat{s} = (\hat{X}, \hat{X}_0, \Sigma, F)$ is a conservative abstraction of $s = (X, X_0, f)$ if the concrete state sets (X and X_0) are covered by the concretization of their abstract representatives ($\hat{X}$ and $\hat{X}_0$) and the transition relation F contains at least all edges induced by the concrete system f [13]. Concretely, three conditions must hold:

1. $X \subseteq \bigcup_{\hat{x} \in \hat{X}} \psi^{-1}(\hat{x})$
2. $X_0 \subseteq \bigcup_{\hat{x}_0 \in \hat{X}_0} \psi^{-1}(\hat{x}_0)$

$$3. \forall x \in X, \exists \sigma \in \Sigma : (\psi(x), \sigma, \psi(f(x))) \in F$$

Conservative LTL lifting. We assume that we are given a safety specification φ of the concrete system s , expressed in linear temporal logic (LTL):

Definition 4 (Linear temporal logic specification). *A linear temporal logic specification is logical formula built from the syntax:*

$$\varphi ::= \Box\phi \mid \Diamond\phi \mid \mathcal{U}\phi \mid \bigcirc\phi \mid \phi_1 \wedge \phi_2 \mid \phi_1 \vee \phi_2 \mid \neg\phi \mid p(x) \mid \top \mid \perp$$

- $\Box, \Diamond, \mathcal{U}$, and $\bigcirc$ denote the globally, eventually, until, and next operators
- $\wedge$ and $\vee$ denote the logical AND and OR operators
- $\neg$ denotes logical negation
- $p : X \rightarrow \{\top, \perp\}$ is an atomic predicate on the state $x \in X$ (e.g., $|x| \geq 3$)
- $\top$ and $\perp$ denote the logical TRUE and FALSE constants

The satisfaction of an LTL specification on a trajectory of the ground-truth system is denoted $\tau(s, x_0) \models \varphi$. Similarly, if $\tau(s, x_0) \models \varphi$ for all $x_0 \in X_0$, then the system satisfies the specification, denoted simply as $s \models \varphi$.

This property is expressed through atomic propositions on the domain X . However, we aim to verify that this property holds on the conservative abstraction $\hat{s}$, which possesses the coarser abstract state space $\hat{X}$. Hence, we must build a *lifted* specification $\hat{\varphi}$ from φ . The satisfaction of the lifted LTL specification $\hat{\varphi}$ on a trajectory of the abstraction resolved under π is denoted $\tau_\pi(\hat{s}, \hat{x}_0) \models \hat{\varphi}$, while the satisfaction of on all abstract trajectories originating from $\hat{x}_0$ is denoted:

$$\tau(\hat{s}, \hat{x}_0) \models \hat{\varphi} \iff \forall \pi \in \Pi, \tau_\pi(\hat{s}, \hat{x}_0) \models \hat{\varphi} \quad (7)$$

To ensure that the verification guarantees on $\hat{s}$ transfer to the concrete system s , we must ensure that $\hat{\varphi}$ is a *conservative LTL translation* of φ :

Definition 5 (Lifted LTL specification). *Given a system s , an LTL safety property φ , and a conservative abstraction $\hat{s}$ induced by ψ , the LTL specification $\hat{\varphi}$ is conservatively lifted to $\hat{s}$ if it holds that*

$$\forall \hat{x}_0 \in \hat{X}_0, x_0 \in \psi^{-1}(\hat{x}_0) : (\tau(\hat{s}, \hat{x}_0) \models \hat{\varphi} \implies \tau(s, x_0) \models \varphi). \quad (8)$$

Equivalently, $\hat{s} \models \hat{\varphi} \implies s \models \varphi$.

This tutorial covers conservative lifting of φ to the system abstraction $\hat{s}$.

Conservative Model Checking. Model checking performs an exhaustive, specification-driven exploration of a model's state space to determine whether that specification holds [4, 7]. To model check the *conservative system abstraction* $\hat{s}$ from an initial abstract state $\hat{x}_0$, an algorithm explores all abstract trajectories that originate from $\hat{x}_0$ under the nondeterminism of the abstraction. Equivalently, it quantifies over all policies $\pi \in \Pi$ that resolve this nondeterminism, yielding a unique trajectory $\tau_\pi(\hat{s}, \hat{x}_0)$ for each π .

The lifted safety specification $\hat{\varphi}$ is satisfied from $\hat{x}_0$ if and only if $\tau_\pi(\hat{s}, \hat{x}_0) \models \hat{\varphi}$ for all $\pi \in \Pi$, which we compactly denote $\tau(\hat{s}, \hat{x}_0) \models \hat{\varphi}$. If there exists any

scheduler $\pi \in \Pi$ such that $\tau_\pi(\hat{s}, \hat{x}_0) \not\models \hat{\varphi}$, then the abstraction is not verified from $\hat{x}_0$; that is, $\tau(\hat{s}, \hat{x}_0) \not\models \hat{\varphi}$. Because the abstraction is conservative, satisfaction on the abstraction implies satisfaction on the concrete system: $\hat{s} \models \hat{\varphi} \implies s \models \varphi$. Consequently, it is impossible for the abstraction to verify the property while the concrete system violates it.

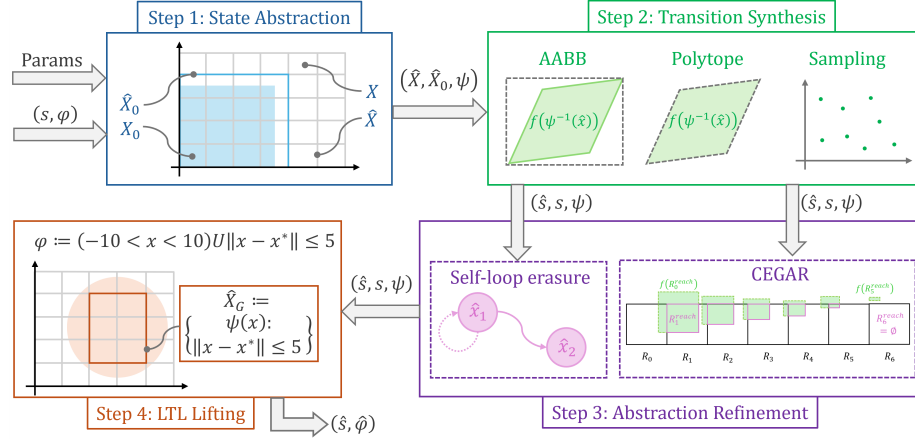


Fig. 1. Overview of the four-step abstraction workflow.

4 Overview of Conservative System Abstraction

We start with a brief overview of the conservative system abstraction pipeline (illustrated in Fig. 1), followed by a running example that we use to exemplify each stage. The abstraction pipeline takes two inputs: (i) a concrete dynamical model (Eq. 1) and (ii) an LTL safety specification (Eq. 4). From these inputs, conservative abstraction proceeds through four steps: (1) abstract the state space, (2) build a conservative transition relation, (3) diagnose and remove spurious behaviors via refinement, and (4) lift the LTL specification to the abstract model. The output is a conservative abstraction of the system in the form of a *finite state machine* (Def. 2) and a lifted specification used in symbolic model checking.

In Step 1 (Sec. 5), we define an *abstraction function* ψ from user-selected hyperparameters (e.g., the coarseness along each dimension), then construct the abstract state set $\hat{X}$ and abstract initial set $\hat{X}_0$ and ensure that Conditions (1) and (2) of Def. 3 are met. In Step 2 (Sec. 6), we compute the abstract transition relation F over $\hat{X}$ so that Condition (3) of Def. 3 holds, and we compare several approaches suited to different classes of dynamics f (affine and nonlinear). In Step 3 (Sec. 8), we identify spurious behaviors (self-loops in F , spurious cycles, and spurious counterexamples) and refine both the state space $\hat{X}$ and transition relation F to eliminate these behaviors without sacrificing soundness. Finally, in Step 4 (Sec. 7), we lift the LTL specification from X to the refined abstract state space $\hat{X}$, ensuring the condition in Def. 5 holds.

Running example. We will use a unicycle model as a running example throughout this tutorial. The unicycle state is given by $x = (y_1, y_2, \theta) \in [0.0, 50.0] \times [0.0, 40.0] \times [-\pi, \pi]$, where y_1 and y_2 represent the two positional coordinates, and θ denotes the system's heading angle. The objective of this system is to reach within 8.0 units of the goal located at $(y_1^{goal}, y_2^{goal}) = (40.0, 20.0)$ while remaining in bounds and at least 5.0 from an obstacle centered at $(y_1^{obs}, y_2^{obs}) = (25.0, 25.0)$. This specification expressed in LTL:

$$\varphi := [(0 < y_1 < 50) \wedge (0 < y_2 < 40) \wedge d_1(y_1, y_2) > 5] \mathcal{U} (d_2(y_1, y_2) \leq 8); \quad (9)$$

$$d_1(y_1, y_2) := \sqrt{(y_1 - 25)^2 + (y_2 - 25)^2}; \quad d_2(y_1, y_2) := \sqrt{(y_1 - 40)^2 + (y_2 - 20)^2}$$

The system dynamics are given by the Dubins model [18]:

$$f(x) = \begin{bmatrix} y_1[k+1] \\ y_2[k+1] \\ \theta[k+1] \end{bmatrix} = \begin{bmatrix} y_1[k] \\ y_2[k] \\ \theta[k] \end{bmatrix} + \begin{bmatrix} v \cos(\theta[k]) \\ v \sin(\theta[k]) \\ u[k] \end{bmatrix} \Delta t, \quad (10)$$

where $\Delta t = 0.5$ is a fixed sampling time, $v = 2.0$ is the constant unicycle speed, and u is the control action (heading rate of change) selected from a deterministic state controller (App. A) designed for goal-attraction and obstacle-repulsion.

5 Step 1: Abstract the State-Space

State-space tessellation. A *finite* state machine (Def. 2) requires a *finite* collection of states to evolve over. However, the concrete model does not fit this template due to the continuous domain X . One solution is to formulate a *state-space abstraction* by imposing a uniform rectilinear grid over X whose induced subsets compose the abstract states $\hat{X}$. Suppose X is a rectangular domain $[\underline{x}^{(1)}, \bar{x}^{(1)}] \times \dots \times [\underline{x}^{(n)}, \bar{x}^{(n)}]$, where $\underline{x}^{(j)}$ and $\bar{x}^{(j)}$ denote the boundaries of the domain along dimension $j \in 1, \dots, n$. Let $c^{(j)}$ be a user-defined count of desired abstract states per dimension j . Then we specify the set of uniformly-spaced breakpoints along each dimension $\mathcal{B} = \{B^{(1)}, \dots, B^{(n)}\}$, where:

$$B^{(j)} = \left(\underline{x}^{(j)} < b_2^{(j)} < \dots < b_{c^{(j)}}^{(j)} < \bar{x}^{(j)} \right), \quad b_2^{(j)} - \underline{x}^{(j)} = \dots = \bar{x}^{(j)} - b_{c^{(j)}}^{(j)}$$

is the tuple of uniformly spaced breakpoints along the j^{th} dimension, including the boundaries. Let $\mathbf{i} = (i_1, \dots, i_n) \in I$ be a vector of indices where $I = \{1, \dots, c^{(1)}\} \times \dots \times \{1, \dots, c^{(n)}\}$ is the valid index set. The induced abstract state set is the collection of abstract states $\hat{x}[\mathbf{i}]$ for all possible indices $\mathbf{i}$, that is, $\hat{X} = \{\hat{x}[\mathbf{i}] \mid \mathbf{i} \in I\} = \{\psi(x) \mid x \in X\}$. The initial abstract state set is also given by $\hat{X}_0 = \{\psi(x_0) \mid x_0 \in X_0\}$, where the abstraction function ψ is defined as

$$x \in \prod_{j=1}^n [b_{i_j}^{(j)}, b_{i_j+1}^{(j)}] \implies \psi(x) = \hat{x}[\mathbf{i}], \quad (11)$$

and its inverse is defined as

$$\psi^{-1}(\hat{x}[\mathbf{i}]) = \{x \in \mathbb{R}^n : b_{i_j}^{(j)} \leq x^{(j)} < b_{i_j+1}^{(j)}, \quad j = 1, \dots, n\}. \quad (12)$$

Both $\hat{X}$ and $\hat{X}_0$ satisfy conditions (1)–(2) of Definition 3.

Running example. Recall the state space of the unicycle system $X = [0.0, 50.0] \times [0.0, 40.0] \times [-\pi, \pi]$. To match the above notation, we represent the state vector as $x = (y_1, y_2, \theta) = (x^{(1)}, x^{(2)}, x^{(3)})$. Suppose we choose $c^{(1)} = c^{(2)} = c^{(3)} = 10$ to be the desired number of abstract state cells along each dimension (10^3 total abstract states). The set of breakpoints is $\mathcal{B} = \{B^{(1)}, B^{(2)}, B^{(3)}\}$, where:

$$B^{(1)} = \{0, 4, \dots, 40\}, \quad B^{(2)} = \{0, 5, \dots, 50\}, \quad B^{(3)} = \left\{-\pi, -\frac{4\pi}{5}, \dots, \pi\right\}$$

In this example, the index i of any abstract state is given by $i = (i_1, i_2, i_3)$ where $i_1, i_2, i_3 \in 1, \dots, 10$. Suppose we want to concretize the abstract state $\hat{x}[3, 2, 1]$. Following Equation 12, we get the subspace:

$$\begin{aligned} \psi^{-1}(\hat{x}[3, 2, 1]) &= \{x \in \mathbb{R}^n : b_3^{(1)} < x^{(1)} < b_4^{(1)}, b_2^{(2)} < x^{(2)} < b_3^{(2)}, b_1^{(3)} < x^{(3)} < b_2^{(3)}\} \\ &= \{x \in \mathbb{R}^n : 8 < x^{(1)} < 12, 5 < x^{(2)} < 10, -\pi < x^{(3)} < 4\pi/5\} \subset X \end{aligned}$$

6 Step 2: Build the Conservative Transitions

With the abstract state space $\hat{X}$ in hand, the next step is to construct the conservative transition relation F induced by the concrete dynamics f . This construction follows a simple deterministic loop over $\hat{X}$ described in Alg. 1:

$$F = \text{BuildTransitions}(\psi, \hat{X}, f)$$

The only nontrivial component of this routine is the procedure **GetSuccessors**, which computes, for each $\hat{x} \in \hat{X}$, a set of candidate successor abstract states consistent with f . This section presents two interchangeable implementations of **GetSuccessors**: (1) a conservative axis-aligned bounding-box (AABB) method (Alg. 2) and (2) a less conservative polytope-based method (Alg. 3). Then, we present a sample-based replacement of **BuildTransitions**, which yields a statistical, not exhaustive, guarantee on the validity of the induced transitions (Alg. 4).

Algorithm 1 BuildTransitions: build the conservative transition relation F

Require: abstraction function $\psi : X \rightarrow \hat{X}$; abstract state space $\hat{X}$; dynamics f

Ensure: transition relation $F \subseteq \hat{X} \times \Sigma \times \hat{X}$

```

1:  $F \leftarrow \emptyset$ 
2:  $\Sigma \leftarrow \{\sigma_{11}, \sigma_{12}, \dots, \sigma_{ij}, \dots\}$  ▷ Alphabet of action labels
3: for all  $\hat{x}_i \in \hat{X}$  do
4:    $\hat{\mathcal{X}} \leftarrow \text{GetSuccessors}(\hat{x}_i, \psi, \hat{X}, f)$ 
5:   for all  $\hat{x}_j \in \hat{\mathcal{X}}$  do
6:      $F \leftarrow F \cup \{(\hat{x}_i, \sigma_{ij}, \hat{x}_j)\}$  ▷  $\sigma_{ij}$  is unique to the ordered pair  $(\hat{x}_i, \hat{x}_j)$ 
7:   end for
8: end for
9: return  $F$ 
```

AABB propagation. We first present a conservative, AABB-based technique for computing the successor set $\hat{\mathcal{X}} \subset \hat{X}$ for an arbitrary state $\hat{x}$ according to ψ and f . Let $R \subset X$ be a region in the n -dimensional state space. An *axis-aligned bounding box* of R is a rectilinear subset of X that inscribes R :

$$\text{AABB}(R) := \prod_{j=1}^n [l^{(j)}, u^{(j)}], \quad l_j = \min_{x \in R} x^{(j)}, \quad u_j = \max_{x \in R} x^{(j)}, \quad (13)$$

where $l^{(j)}$ and $u^{(j)}$ are the states in R containing the min/max j th component.

To conservatively approximate the successors of $\hat{x}$, we compute the AABB of its concrete image, and then determine the set of states that this image over-approximation intersects with. Let $f(\psi^{-1}(\hat{x}))$ be the concrete image of $\hat{x}$ under f . Then, its AABB over-approximation is denoted $\text{AABB}(f(\psi^{-1}(\hat{x})))$. To build the successor set, Alg. 2 iterates over all candidate successors $\hat{x} \in \hat{X}$ and determines whether their concretization intersects with this bounding box. If so, this is allocated as a transition relation in F . This technique, paired with Alg. 1, yields a transition relation that satisfies condition (3) of Def. 3.

Algorithm 2 GetSuccessorsAABB: compute successors of $\hat{x}$ according to the AABB image over-approximation

Require: abstract state $\hat{x}$; abstraction function $\psi : X \rightarrow \hat{X}$; abstract state space $\hat{X}$; dynamics f

Ensure: successor set $\hat{\mathcal{X}} \subseteq \hat{X}$

- 1: $\hat{\mathcal{X}} \leftarrow \emptyset, R \leftarrow \psi^{-1}(\hat{x})$
- 2: $R' \leftarrow \text{AABB}(f(R))$ ▷ Over-approximate image of R via AABB (Eqn. 13)
- 3: **for all** $\hat{x}' \in \hat{X}$ **do** ▷ Iterate over candidate successors
- 4: **if** $\psi^{-1}(\hat{x}') \cap R'$ **then**
- 5: $\hat{\mathcal{X}} \leftarrow \hat{\mathcal{X}} \cup \{\hat{x}'\}$
- 6: **end if**
- 7: **end for**
- 8: **return** $\hat{\mathcal{X}}$

Polytope propagation. AABBs over-approximate the image of the concretized abstract state relatively tightly when f is linear. However, in the nonlinear setting, the mapping from a concretized abstract state to its image through f may involve shear, stretching, and even folding. Hence, an AABB may be too conservative and lead to spurious edges in F . A less conservative technique is to over-approximate the image with a polytope PT:

$$\text{PT}(R) := \left\{ \sum_{k=1}^{2^n} \lambda_k \mid x \in R, \lambda_k \geq 0, \sum_{k=1}^{2^n} \lambda_k = 1 \right\}, \quad (14)$$

where λ_k are convex-combination coefficients used to form the convex hull over R . We denote the polytope-based image over-approximation of $\hat{x}$ as $\text{PT}(f(\psi^{-1}(\hat{x})))$.

To build the successor set using the polytope over-approximation of $f(\psi^{-1}(\hat{x}))$, Alg. 3 first rules out obvious non-intersecting subsets $\psi^{-1}(\hat{x})$ using the bounding box over-approximation $\text{AABB}(f(\psi^{-1}(\hat{x})))$ (which inscribes the polytope), and then iterates over the remaining states to build the successor set. This greatly improves the performance of the algorithm since AABB intersection tests are faster than the polytope tests. Like the AABB method, this technique yields a transition relation that satisfies condition (3) of Def. 3.

Algorithm 3 GetSuccessorsPT: compute successors of $\hat{x}$ using polytope over-approximation with AABB culling

Require: abstract state $\hat{x}$; abstraction function $\psi : X \rightarrow \hat{X}$; abstract state space $\hat{X}$; dynamics f ;
Ensure: successor set $\hat{\mathcal{X}} \subseteq \hat{X}$

- 1: $\hat{\mathcal{X}} \leftarrow \emptyset$, $R \leftarrow \psi^{-1}(\hat{x})$
- 2: $R'_{\text{AABB}} \leftarrow \text{AABB}(f(R))$ ▷ Over-approximate image of R via AABB (Eqn. 13)
- 3: $R'_{\text{PT}} \leftarrow \text{PT}(f(R))$ ▷ Over-approximate image of R via PT (Eqn. 14)
- 4: $\mathcal{C} \leftarrow \emptyset$ ▷ Define set of filtered candidates
- 5: **for all** $\hat{x}' \in \hat{X}$ **do**
- 6: **if** $\psi^{-1}(\hat{x}') \cap R'_{\text{AABB}}$ **then**
- 7: $\mathcal{C} \leftarrow \mathcal{C} \cup \{\hat{x}'\}$ ▷ Allocate as a potential candidate
- 8: **end if**
- 9: **end for**
- 10: **for all** $\hat{x}' \in \mathcal{C}$ **do**
- 11: **if** $\psi^{-1}(\hat{x}') \cap R'_{\text{PT}}$ **then**
- 12: $\hat{\mathcal{X}} \leftarrow \hat{\mathcal{X}} \cup \{\hat{x}'\}$
- 13: **end if**
- 14: **end for**
- 15: **return** $\hat{\mathcal{X}}$

Sampling-based successors. When the dynamics f are strongly nonlinear and/or the rectilinear tessellation is coarse, sampling can yield the least conservative approximation of the successor set $\hat{\mathcal{X}}$. The trade-off is that, unlike the above propagation methods, sampling may fail to observe a genuine transition, feasible in the concrete system. Consequently, the transition relation F produced by sampling does not deterministically satisfy Condition (3) of Def. 3.

Instead, one can make a probabilistic guarantee of the following form: after N samples, F contains all ground-truth transitions with confidence at least $1 - \alpha$. To make this precise, we first describe a “ground-truth” guarantee expressed in terms of the true transition relation (which we denote F^*) and then use it as a template for the data-driven setting where F^* is unknown and only sampled observations of f are available. Alg. 4 describes this data-driven method for building and certifying F , replacing Alg. 1.

Ground-truth guarantee. Consider a uniform distribution over the state space X , denoted $\text{Unif}(X)$. This induces a categorical distribution $\text{Cat}(F^*)$ over transi-

Algorithm 4 `SampleTransitions`: build F by sampling from $\text{Unif}(X)$ until the certificate condition (Eq. 17) is met

Require: abstraction function $\psi : X \rightarrow \hat{X}$; abstract state space $\hat{X}$; state space X ; dynamics f ; confidence level δ ; significance level β

Ensure: transition relation $F \subseteq \hat{X} \times \Sigma \times \hat{X}$

```

1:  $F \leftarrow \emptyset$ ,  $\Sigma \leftarrow \{\sigma_{11}, \sigma_{12}, \dots, \sigma_{ij}, \dots\}$ 
2:  $\overline{M}_N \leftarrow 1.0$ ,  $N \leftarrow 0$ ,  $c_1 \leftarrow 0$ 
3:  $C[\cdot] \leftarrow \emptyset$  ▷ Map: triple  $\mapsto$  transition allocation count
4: while  $\overline{M}_N \geq \beta$  do
5:    $x \leftarrow \text{Unif}(X)$ ,  $\hat{x}_i \leftarrow \psi(x)$ ,  $\hat{x}_j \leftarrow \psi(f(x))$ 
6:    $t \leftarrow (\hat{x}_i, \sigma_{ij}, \hat{x}_j)$  ▷  $\sigma_{ij}$  unique to ordered pair  $(\hat{x}_i, \hat{x}_j)$ 
7:   if  $t \notin \text{dom}(C)$  then
8:      $C[t] \leftarrow 1$ ,  $c_1 \leftarrow c_1 + 1$ ,  $F \leftarrow F \cup \{t\}$ 
9:   else if  $C[t] = 1$  then
10:     $C[t] \leftarrow 2$ ,  $c_1 \leftarrow c_1 - 1$ 
11:   else
12:     $C[t] \leftarrow C[t] + 1$ 
13:   end if
14:    $N \leftarrow N + 1$ 
15:    $\overline{M}_N \leftarrow \frac{c_1}{N} + (2\sqrt{2} + \sqrt{3})\sqrt{\frac{\ln(3/\delta)}{N}}$  ▷ (Eq. 16)
16: end while
17: return  $F$ ,  $N$ ,  $\overline{M}_N$ 

```

tion relations $(\psi(x), \sigma, \psi(f(x)))$ in F^* with categories $\ell = \{1, \dots, K\}$, where $K = |F^*|$ is the number of ground-truth transitions in F^* . Each triple $(\psi(x), \sigma, \psi(f(x)))$ with index ℓ has a probability mass q_ℓ in this $\text{Cat}(F^*)$. Intuitively, q_ℓ is the chance that after a single draw $x \sim \text{Unif}(X)$, the algorithm allocates the relation $(\psi(x), \sigma, \psi(f(x)))$ to F . The exact formula for q_ℓ can be found in App. A.

We are ultimately interested in the probability that N i.i.d. draws suffice to recover *all* true transitions in F^* (precisely, $F = F^*$). By the inclusion-exclusion principle, this probability is given by:

$$\Pr(F^* = F) = \sum_{\forall L \subseteq \{1, \dots, K\}} (-1)^{|L|} \left(1 - \sum_{\ell \in L} q_\ell \right)^N = 1 - \alpha, \quad (15)$$

where L is a subset of the category labels. Finally, observing all transitions implies that every concrete one-step behavior is represented in the abstract relation, so $F^* = F$ coincides with satisfaction of Condition (3) in Def. 3. Hence, Condition (3) is satisfied with confidence $1 - \alpha$.

Probably-approximately correct (PAC) guarantee. In practice, we do not have access to the parameters of the induced categorical distribution $\text{Cat}(F^*)$ (q_ℓ and even the support size K are unknown). Nevertheless, we can still sample transition relations $(\psi(x), \sigma, \psi(f(x)))$ from this categorical distribution $\text{Cat}(F^*)$ by sampling from $\text{Unif}(X)$.

Let $F_N \subseteq F^*$ denote the set of relations discovered after drawing N samples from $\text{Cat}(F^*)$. In this distribution-agnostic setting, it is impossible to compute $\Pr(F_N = F^*)$, since the absence of a relation in F_N that exists in F^* does not distinguish “impossible” from “possible but rare.” Indeed, if some p_ℓ is extremely small, then $\Pr(F_N = F^*)$ is negligible for feasible N .

Therefore, to obtain a practical guarantee, one can adopt a PAC relaxation: consider only the transitions in F^* with probabilities above a user-chosen significance level, say $\beta \in (0, 1)$. We call a transition β -significant if $p_\ell \geq \beta$. Rather than certifying that *every* transition has been observed, we certify that the unobserved probability mass is small. This is captured by the *missing mass* M_N , defined as the total probability of relations not allocated to F_N , i.e., $M_N = \Pr(F^* \setminus F_N)$. The missing mass is a classical object in distribution learning [5]; operationally, it upper-bounds the probability that the next draw yields a previously unseen label. One estimator of M_N is the Good–Turing estimate [22]: $M_N \approx \frac{c_1}{N}$, where c_1 is the number of labels in F_N that appear exactly once.

Since this estimator is not necessarily conservative, McAllester and Schapire [28] provided an upper bound on the missing mass. With probability at least $1 - \delta$,

$$M_N \leq \frac{c_1}{N} + (2\sqrt{2} + \sqrt{3})\sqrt{\frac{\ln(3/\delta)}{N}} = \overline{M}_N. \quad (16)$$

Thus, if $\overline{M}_N < \beta$, then $M_N < \beta$ holds with confidence at least $1 - \delta$, i.e.,

$$\Pr(M_N < \beta) \geq \Pr(M_N \leq \overline{M}_N) \geq 1 - \delta, \quad \overline{M}_N < \beta. \quad (17)$$

This certificate has a direct interpretation for abstraction soundness under sampling. If $M_N < \beta$, then the total probability mass of unobserved transitions is below β , which implies that every β -significant transition (every region with $p_\ell \geq \beta$) must have been observed and therefore added to F_N . Equivalently, with probability at least $1 - \delta$, the sampled relation F_N contains all abstract transitions that occur with probability at least β under uniform sampling $\text{Unif}(X)$.

The parameter β therefore controls the approximation level. Choosing a smaller β tightens the guarantee (at the cost of requiring larger N), and the ground-truth guarantee can even be recovered if β falls below the minimum categorical probability. Without access to F^* , however, the strongest data-driven guarantee we can certify in general is coverage of all β -significant transitions.

Running example. We illustrate the sampling-based construction of the transition relation and its PAC-style coverage certificate on the running unicycle example. Recall the state space $X = [0.0, 50.0] \times [0.0, 40.0] \times [-\pi, \pi]$. We draw $N = 10000$ samples $x \sim \text{Unif}(X)$ and, for each sample, record the induced abstract transition $(\psi(x), \psi(f(x)))$. For example, two sampled states might be $x_1 = (19.5, 23.8, -0.1)$ and $x_2 = (12.1, 9.6, -0.73)$, which abstract to $\psi(x_1) = \hat{x}[5, 5, 5]$ and $\psi(x_2) = \hat{x}[4, 2, 3]$. Then, determine the successor abstract states:

$$\begin{aligned} \psi(x'_1) &= \hat{x}[6, 6, 5], & x'_1 &= f(x_1) \approx (21.5, 23.6, -0.09) \\ \psi(x'_2) &= \hat{x}[4, 2, 3], & x'_2 &= f(x_2) \approx (13.6, 8.3, -0.65) \end{aligned}$$

These two samples thus contribute the transitions $(\hat{x}[5, 5, 5], \sigma, \hat{x}[6, 6, 5])$ and $(\hat{x}[4, 2, 3], \sigma, \hat{x}[4, 2, 3])$ to the sampled relation F_N . Repeat this procedure over all 10000 samples.

To certify coverage, fix a significance threshold $\beta = 0.1$ and confidence parameter $\delta = 0.05$. Let c_1 denote the number of transition labels that appear *exactly once* among the N observations; suppose $c_1 = 12$. The McAllester–Schapire upper bound on the missing mass is then

$$\begin{aligned} \overline{M}_N &= \frac{c_1}{N} + (2\sqrt{2} + \sqrt{3}) \sqrt{\frac{\ln(3/\delta)}{N}} \\ &= \frac{12}{10000} + (2\sqrt{2} + \sqrt{3}) \sqrt{\frac{\ln(3/0.05)}{10000}} \approx 0.0935 < \beta, \end{aligned}$$

which enables the PAC guarantee $\Pr(M_N < 0.1) \geq 0.95$. In words, after $N = 10000$ samples we can assert (with at least 95% confidence) that the total probability mass of *unobserved* transitions is below 0.1, and hence that F_N contains every transition that occurs with probability at least 10% under uniform sampling from X .

7 Step 3: Purge Degenerate Behaviors

Types of degenerate behaviors. Assured approximation of the ground-truth model (either through over-approximation or sampling) paired with *conservative model checking* ensures that guarantees made on the abstraction transfer to the ground-truth system (Def. 3), i.e., $s \models \varphi \iff \hat{s} \models \hat{\varphi}$. However, conservative abstraction often produces *degenerate behaviors* that can yield pessimistic outcomes in which the abstraction violates the lifted specification even though the concrete system satisfies it (i.e., $\hat{s} \not\models \hat{\varphi}$ but $s \models \varphi$). In this section, we identify two common classes of degenerate behaviors found in conservative abstractions – *spurious transitions* and *self-loop transitions* – then discuss *assured self-loop erasure* and *CEGAR* to purge them from the abstraction.

Spurious transitions. A *spurious transition* is a relation between two abstract states in F that cannot occur in the ground-truth system. Formally, it is a relation $(\hat{x}, \sigma, \hat{x}')$ such that there is no mapping from $\psi^{-1}(\hat{x})$ to $\psi^{-1}(\hat{x}')$ through f , i.e., the image $f(\psi^{-1}(\hat{x}))$ is disjoint from $\psi^{-1}(\hat{x}')$.

Spurious transitions arise from conservative methods for building the transition relation F . These transitions appear when the set over-approximation of the image $f(\psi^{-1}(\hat{x}))$ (AABB($\cdot$) or Poly($\cdot$)) intersects with $\psi^{-1}(\hat{x}')$, while the ground-truth image does not. These transitions are harmful in conservative model checking because they can compound into spurious *counterexample trajectories* — trajectories of the abstract system that do not satisfy $\hat{\varphi}$ and are impossible to traverse in the concrete system.

Self-loop transitions. A *self loop* is a transition in F that maps an abstract state $\hat{x}$ to itself, formally, $(\hat{x}, \sigma, \hat{x}) \in F$. Self-loop transitions may be *spurious transitions* that are artifacts of conservative successor procedures (AABB- or

polytope-based methods), but they may also be an authentic representation of the ground-truth dynamics in $\psi^{-1}(\hat{x})$. The underlying issue is that the induced finite-state model is memoryless: it does not distinguish concrete states that are “about to exit” the cell from those that will remain inside for at least one more step. As a result, the abstraction admits the possibility of remaining in $\hat{x}$ for infinitely many steps, since a scheduler may repeatedly select the self-loop.

This is particularly harmful when reasoning about liveness specifications. Since model checking considers worst-case scheduling, it will return the infinite self-loop trace $(\hat{x}, \hat{x}, \dots)$ as a counterexample of a liveness specification, unless the trace trivially satisfies the specification (e.g., the state already lies in a goal set that must eventually be reached). For many applications, it is best to remove such self-loops by certifying that the concrete system can escape the cell in finite time. Spurious self-loop transitions can be removed through *CEGAR*, while both spurious and non-spurious ones can be removed through *direct certification*.

Counterexample-guided abstraction refinement. Compounding spurious transitions (spurious trajectories) are an artifact of successor over-approximation (AABB- and polytope-based) and may lead to *spurious counterexamples*. Concretely, these counterexamples are deterministic abstract trajectories $\tau_\pi(\hat{s}, \hat{x}_0)$ that fail to satisfy $\hat{\varphi}$. Counterexample-guided abstraction refinement (CEGAR) is an iterative procedure with in-the-loop verification that purges spurious transitions in F along the counterexample trajectory by proving that such a transition cannot occur within the ground-truth system. The algorithm locally refines state cells $\hat{x} \in \hat{X}$ by splitting cells until the counterexample is refuted, or a stopping condition has been met (allowable refinements). Below, we explain the main operations in CEGAR (counterexample validation and refinement), and direct the readers to the original application of CEGAR to dynamical systems [13] for further details.

Counterexample validation. A counterexample trajectory $\tau_\pi(\hat{s}, \hat{x}_0) = (\hat{x}_0, \dots, \hat{x}_M)$ is an M -length trajectory output by the model checker when $\tau_\pi(\hat{s}, \hat{x}_0) \not\models \hat{\varphi}$. A critical step is determining whether this trajectory is spurious; that it, if for all $x_0 \in \psi^{-1}(\hat{x}_0)$, $\tau(s, x_0) \models \varphi$; i.e., all ground-truth trajectories originating from the subspace $\psi^{-1}(\hat{x}_0)$ do satisfy the LTL specification φ .

Let $R_k = \psi^{-1}(\hat{x}_k)$ be the concretized region of $\hat{x}_k$ for $k \in \{1, \dots, M\}$. The reachable part of R_k from R_{k-1} is given by:

$$R_0^{reach} = R_0 = \psi^{-1}(\hat{x}_0), \quad R_k^{reach} = R_k \cap f(R_{k-1}^{reach}), \quad k = 1, \dots, M \quad (18)$$

If $R_k^{reach} = \emptyset$ for some k , then the abstract prefix cannot be realized concretely, and the counterexample trajectory is spurious. Intuitively, this recursion along $\tau_\pi(\hat{s}, \hat{x}_0)$ searches for the first instance (from the initial state) where the concrete reachable set diverges from the counterexample trajectory (see Fig. 2).

Refinement operations. When $\tau_\pi(\hat{s}, \hat{x}_0)$ is found to be spurious, CEGAR refines the tessellation to block the refuted behaviors while preserving abstraction conservatism [13]. Let k be the first index such that $R_k = \emptyset$ in the validation recursion. Refinement then targets the transition $(\hat{x}_{k-1}, \hat{x}_k)$ by *splitting* $\hat{x}_k$ into

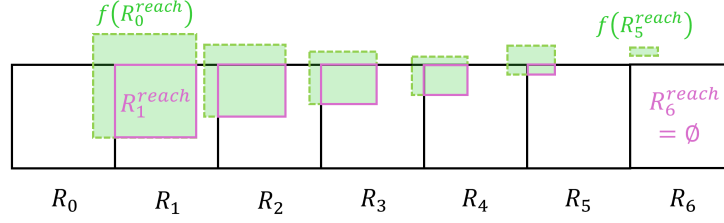


Fig. 2. A counterexample trajectory admits the sequence of concrete regions $(R_0, R_1, \dots)$. Counterexample validation iteratively determines the residual intersecting area between the concretized trajectory and its abstract counterpart. In this example, this area is 0 by $k = 6$, validating that the counterexample trajectory is indeed spurious.

a reachable subset and its complement (equivalently, splitting $\psi(\cdot)$ so these subsets become distinct abstract states) until the counterexample is refuted or the stopping condition has been met.

Assured self-loop erasure. Successor-approximation procedures may introduce self-loop transitions of the form $(\hat{x}, \sigma, \hat{x})$ into F . Regardless of whether these transitions are spurious or authentic, they often lead to poor verification outcomes when we attempt to verify liveness specifications through conservative model checking. It is therefore reasonable to purge a self-loop from F if we can certify a *finite-time escape* property for the associated concretization. In other words, if we can show that every concrete trajectory originating in $\psi^{-1}(\hat{x})$ exits $\psi^{-1}(\hat{x})$ within K steps, then we may purge the self loop. We present two techniques for certifying self-loop erasure: (1) an iterative bounding-box approximation of the propagated reachable set and (2) a sampling-based strategy with an associated PAC guarantee.

Reachable-set propagation. Let $R_0 = \psi^{-1}(\hat{x})$ be the initial concretized region of $\hat{x}$. Let $f(R_k)$ denote the image of R_k under f . The propagation of R_k over $k = 1, \dots, K$ is given by:

$$R_{k+1} = R_k \cap AAB B(f(R_k)). \quad (19)$$

Intuitively, R_{k+1} is exactly the set of states in R_k whose next-step image can also lie in R_k . Repeated intersection therefore filters out states that must escape, so if we find some $k \in \{1, \dots, K\}$ such that $R_k = \emptyset$, then no concrete trajectory can remain in $\psi^{-1}(\hat{x})$ for $k \leq K$ steps.

Sampling-based with PAC guarantee. Suppose we impose a uniform distribution $x_0 \sim \text{Unif}(\psi^{-1}(\hat{x}))$. This induces a distribution over ground truth trajectories $\tau \sim \text{Pr}(\tau(s, x_0))$. Denote the K -length trajectory drawn from this distribution as $\tau_{1:K} = (x_0, \dots, x_K)$. Define the event $E = \{\exists k \in \{1, \dots, K\} : \tau_k \notin \psi^{-1}(\hat{x})\}$ to mean that the K -length trajectory $\tau_{1:K}$ has escaped $\psi^{-1}(\hat{x})$. We denote the (unknown) probability of this event as $q_\tau = \text{Pr}(E)$.

Suppose we draw N of these K -length trajectories $\tau \sim \Pr(\tau(s, x_0))$. If we observe that E occurs N times, then we assert the Binomial PAC guarantee:

$$\Pr\left(q_\tau \geq \gamma^{1/N}\right) \geq 1 - \gamma \quad (20)$$

In words: with $1 - \gamma$ confidence, the chance that any K -length trajectory escapes $\psi^{-1}(\hat{x})$ is at least $\gamma^{1/N}$. In this distribution-agnostic setting (analogous to the sample-based successor guarantee), the best we can guarantee is the coverage of successful trajectories. If we observe any $\neg E$ after N trials, then we have disproven absolute coverage, and cannot remove the self loop.

Running example. We exemplify the sample-based self-loop erasure certificate on the running unicycle example. Suppose the constructed relation F contains a self-loop $(\hat{x}[5, 5, 5], \sigma, \hat{x}[5, 5, 5])$. Per Eq. 13, the corresponding concretization is

$$\psi^{-1}(\hat{x}[5, 5, 5]) = [16, 20] \times [20, 25] \times [-\pi/10, 0].$$

Fix $K = 10$ and a target confidence parameter $\gamma = 0.1$. We draw $N = 100$ initial conditions uniformly from $\psi^{-1}(\hat{x}[5, 5, 5])$ and simulate each trajectory for at most K steps, declaring the certificate successful if every rollout exits the cell within K steps. For instance, one draw yields $x_0 = (18.5, 22.8, -0.1)$, and propagating through the unicycle dynamics (Eq. 10) gives

$$x_1 = f(x_0) = \begin{bmatrix} 18.5 \\ 22.8 \\ -0.1 \end{bmatrix} + \begin{bmatrix} 2 \cos(-0.1) \\ 2 \sin(-0.1) \\ 0.01 \end{bmatrix} \approx \begin{bmatrix} 20.5 \\ 22.6 \\ -0.09 \end{bmatrix}. \quad (21)$$

which lies outside $\psi^{-1}(\hat{x}[5, 5, 5])$ after just one step. Observing the same behavior for all $N = 100$ rollouts, we conclude the PAC-style guarantee

$$\Pr\left(p_\tau \geq 0.1^{1/10}\right) \geq 1 - 0.1 \quad \equiv \quad \Pr(p_\tau \geq 0.79) \geq 0.9.$$

That is, given that all 100 sampled trajectories exit within 10 steps, we certify with 90% confidence that at least 79% of trajectories starting in $\psi^{-1}(\hat{x}[5, 5, 5])$ also escape within K steps.

8 Step 4: Lift the LTL Specification to the Abstraction

With a refined conservative abstraction $\hat{s}$ in place, the remaining task is to lift the concrete LTL specification φ to an abstract specification $\hat{\varphi}$ over $\hat{X}$ so that conservative model checking is sound, i.e., $\hat{s} \models \hat{\varphi} \implies s \models \varphi$ (and by contraposition, $\hat{s} \not\models \hat{\varphi} \iff s \not\models \varphi$). This section introduces the additional temporal-logic machinery needed for lifting, defines a homomorphic translation ρ that maps φ to $\hat{\varphi}$, and then illustrates this lifting on the unicycle example.

May and must predicates. Let AP denote the set of atomic propositions p appearing in φ , where $p : X \rightarrow \{\top, \perp\}$. The truth set of an atom is $\llbracket p \rrbracket := \{x \in$

$X \mid p(x) = \top\}$. Given an abstract state $\hat{x} \in \hat{X}$ and its concretization $\psi^{-1}(\hat{x})$, we define *may* and *must* satisfaction of p on $\hat{x}$ by

$$\text{May}_p(\hat{x}) \iff \psi^{-1}(\hat{x}) \cap \llbracket p \rrbracket \neq \emptyset, \quad \text{Must}_p(\hat{x}) \iff \psi^{-1}(\hat{x}) \subseteq \llbracket p \rrbracket. \quad (22)$$

Intuitively, p *may* hold on $\hat{x}$ iff at least one concrete state in $\psi^{-1}(\hat{x})$ satisfies p , whereas p *must* hold on $\hat{x}$ iff every concrete state in $\psi^{-1}(\hat{x})$ satisfies p . We lift these to atomic predicates to the abstract state space such that:

$$\hat{x} \models p^{\text{May}} \iff \text{May}_p(\hat{x}), \quad \hat{x} \models p^{\text{Must}} \iff \text{Must}_p(\hat{x}). \quad (23)$$

Homomorphic translation. Assume φ is in negation normal form. We define a homomorphic translation operator $\rho(\cdot)$ where $\hat{\varphi} = \rho(\varphi)$. We explicitly define the operations of this function, reusing the syntax from Def. 4:

$$\rho(p) := p^{\text{Must}}, \quad \rho(\neg p) := \neg p^{\text{May}}, \quad (24)$$

$$\rho(\phi_1 \wedge \phi_2) := \rho(\phi_1) \wedge \rho(\phi_2), \quad \rho(\phi_1 \vee \phi_2) := \rho(\phi_1) \vee \rho(\phi_2), \quad (25)$$

$$\rho(\Box \phi) := \Box \rho(\phi), \quad \rho(\Diamond \phi) := \Diamond \rho(\phi), \quad (26)$$

$$\rho(\phi_1 \mathcal{U} \phi_2) := \rho(\phi_1) \mathcal{U} \rho(\phi_2), \quad \rho(\bigcirc \phi) := \bigcirc \rho(\phi). \quad (27)$$

The temporal operators are preserved structurally, while atomic propositions are strengthened or weakened via must/may to account for state aggregation. Iterating these rules over the syntax of φ yields a lifted specification $\hat{\varphi}$ in $\hat{X}$ which, by construction, is sound for conservative model checking, i.e., $\hat{s} \models \hat{\varphi} \implies s \models \varphi$.

Running example. Recall the concrete LTL specification from Eqn. 9:

$$\varphi := [(0 < y_1 < 50) \wedge (0 < y_2 < 40) \wedge d_1(y_1, y_2) > 5] \mathcal{U} (d_2(y_1, y_2) \leq 8)$$

For compactness, define $p_1 := (0 < y_1 < 50)$, $p_2 := (0 < y_2 < 40)$, $p_3 := (d_1(y_1, y_2) > 5)$, and $p_4 := (d_2(y_1, y_2) \leq 8)$. We lift the concrete specification by applying the above translation rules:

$$\begin{aligned} \hat{\varphi} &= \rho(\varphi) = \rho([p_1 \wedge p_2 \wedge p_3] \mathcal{U} [p_4]), \\ &= \rho(p_1 \wedge p_2 \wedge p_3) \mathcal{U} \rho(p_4) \quad (\text{via Eq. 27}), \\ &= \rho(p_1) \wedge \rho(p_2) \wedge \rho(p_3) \mathcal{U} \rho(p_4) \quad (\text{via Eq. 25}), \\ &= [p_1^{\text{Must}} \wedge p_2^{\text{Must}} \wedge p_3^{\text{Must}}] \mathcal{U} p_4^{\text{Must}} \quad (\text{via Eq. 24}) \end{aligned}$$

Here p_k^{Must} is the *must* lift of the atomic predicate p_k to $\hat{X}$, defined by

$$\hat{x} \models p_k^{\text{Must}} \iff \text{Must}_{p_k}(\hat{x}) \iff \psi^{-1}(\hat{x}) \subseteq \llbracket p_k \rrbracket.$$

As a concrete illustration, consider p_1 with truth set $\llbracket p_1 \rrbracket := \{(y_1, y_2, \theta) \in X \mid 0 < y_1 < 50\}$. The abstract states on which p_1 holds in the must sense are

$$\hat{X}_{p_1^{\text{Must}}} := \{\hat{x} \in \hat{X} \mid \psi^{-1}(\hat{x}) \subseteq \llbracket p_1 \rrbracket\}.$$

Intuitively, $\hat{X}_{p_1^{\text{Must}}}$ consists of abstract states whose concretizations that do not include the boundary states. Thus, whenever $\hat{x} \in \hat{X}_{p_1^{\text{Must}}}$ we have $\hat{x} \models p_1^{\text{Must}}$, and therefore p_1 holds for every concrete state in $\psi^{-1}(\hat{x})$.

9 Illustrative Case Studies

The purpose of our experimental evaluation is threefold: (i) demonstrate the four-step abstraction pipeline, (ii) reveal the tradeoff between transition-building subroutine runtime and verification performance, and (iii) illustrate the improved performance after removing degenerate behaviors. For each case study and each technique, we abstract the state space, conservatively construct the transition relation via Algorithm 1, purge degenerate behaviors from the abstraction, lift the LTL specification from the concrete system to the system abstraction, and then model check the resulting abstractions against their respective LTL specifications using `pyModelChecking` [8].

9.1 System Descriptions

Synthetic System. We utilize a synthetic, linear time-invariant, discrete-time, and globally exponentially stable (GES) system with two dimensions:

$$x[k+1] = x^* + A(x[k] - x^*), \quad A = \begin{bmatrix} 0.8 & -0.3 \\ 0.3 & 0.8 \end{bmatrix}, \quad x^* = \begin{bmatrix} 5.0 \\ 5.0 \end{bmatrix}$$

The state space of this synthetic dynamical system is given by $x = (x_1, x_2) \in X = (-10, 10) \times (-10, 10)$. The control objective is to converge to within 2.0 units of the equilibrium point x^* within a finite number of steps while always remaining inside of X . The specification is translated into LTL as:

$$\varphi := [(-10 < x_1 < 10) \wedge (-10 < x_2 < 10)] \mathcal{U} [\|x - x^*\| \leq 2.0] \quad (28)$$

Mountain Car. We utilize the OpenAI Gym [6] MountainCar-v0 benchmark — a widely adopted testbed for reinforcement learning — in our evaluation. This environment models a two-dimensional control problem with state variables $x = (y, v) \in [-1.2, 0.6] \times [-0.07, 0.07]$, where y is the horizontal position of the vehicle and v is its velocity. At each timestep, the agent selects from actions $u \in \{0, 1, 2\}$, corresponding respectively to the application of a negative force (throttle leftward), no force, or a positive force (throttle rightward).

The control objective of the Mountain Car is to reach the top of a steep incline ($y \geq 0.5$). For our experiments, we trained a lightweight Deep Q-Network (DQN) policy over 5000 episodes with $\epsilon_0 = 1.0$ and a decay rate of 0.9983. During closed-loop execution with non-linear dynamics, the policy is guided by ground-truth position estimates drawn from the environment at runtime. This specification can be translated into LTL as $\varphi := \Diamond(y > 0.5)$.

Autonomous Unicycle. Sec. 4 contains the details on the unicycle case study.

9.2 Evaluation Metrics

Abstraction metrics. Prior to model checking, we compute several descriptive metrics for the conservative abstraction $\hat{s} = (\hat{X}, \hat{X}_0, \Sigma, F)$. These metrics

quantify *degenerate behaviors* in the abstract transition system (spurious transitions and self-loop), which are often correlated with pessimistic verification outcomes. Let $\text{Succ}(\hat{x}) := \{\hat{x}' \in \hat{X} \mid \exists \sigma \in \Sigma : (\hat{x}, \sigma, \hat{x}') \in F\}$ denote the successor set of $\hat{x}$. We report the *self-loop proportion* $\text{SLP} := |F_{\text{self}}|/|\hat{X}|$, where $F_{\text{self}} := \{(\hat{x}, \sigma, \hat{x}') \in F \mid \hat{x} = \hat{x}'\}$, along with basic successor statistics: the *mean successor count* $\text{mSu} := \frac{1}{|\hat{X}|} \sum_{\hat{x} \in \hat{X}} |\text{Succ}(\hat{x})|$, the *minimum and maximum successor counts* $\underline{\text{Su}} := \min_{\hat{x} \in \hat{X}} |\text{Succ}(\hat{x})|$ and $\overline{\text{Su}} := \max_{\hat{x} \in \hat{X}} |\text{Succ}(\hat{x})|$, and the *standard deviation* $\text{Su}_\sigma := \sqrt{\frac{1}{|\hat{X}|} \sum_{\hat{x} \in \hat{X}} (|\text{Succ}(\hat{x})| - \text{mSu})^2}$.

Verification metrics. In each experiment, we treat the entire abstract state space as the set of initial states (i.e., $\hat{X}_0 = \hat{X}$) and then check the satisfaction of the LTL specification on the system under every scheduler $\pi \in \Pi$ with the initial state state ($\hat{s} \models \varphi$), yielding the satisfying subset of initial states $\hat{X}_{\text{sat}} \subseteq \hat{X}_0$.

We obtain the “ground truth” satisfaction of φ from the concrete system through lightweight reachability. We partition the state space X with a uniform grid, inducing an abstract state space $\tilde{X}$. We denote the concretization of $\tilde{x} \in \tilde{X}$ as $\psi^{-1}(\tilde{x}) = \{x \mid \tilde{\psi}(x) = \tilde{x}\}$. This reachability returns the subset of abstract states $\tilde{X}_{\text{safe}} \subseteq \tilde{X}$ that likely satisfy the LTL specification.

We present the results of verification on each method through several standard metrics: the *build time*, which is the runtime of all abstraction steps, and *verification time*, which is the total time of model checking the final abstraction.

The *true positive rate* (TPR) measures the proportion of truly safe states in $\hat{X}$ that were verifiably safe:

$$\text{TPR} := \frac{\sum_{\hat{x} \in \hat{X}_{\text{sat}}} \mathbb{I} \left[\psi^{-1}(\hat{x}) \subset \bigcup_{\tilde{x} \in \tilde{X}_{\text{safe}}} \tilde{\psi}^{-1}(\tilde{x}) \right]}{\sum_{\hat{x} \in \hat{X}} \mathbb{I} \left[\psi^{-1}(\hat{x}) \subset \bigcup_{\tilde{x} \in \tilde{X}_{\text{safe}}} \tilde{\psi}^{-1}(\tilde{x}) \right]} \quad (29)$$

9.3 Experimental Results

Model size ablation. We performed a small ablation on all combinations of case studies and transition-building subroutines to analyze the effect of abstraction granularity on model checking performance. In particular, we vary the size of the abstract state space along each dimensions, and report the verification results in TPR. For instance, a size of 100 abstract states along each of the n state space dimensions yields $|\hat{X}| = 100^n$.

For each case study and transition-building subroutine, we report TPR over a small window of these dimension sizes. Fig. 3 reveals that granular abstractions do indeed yield higher TPR. Furthermore, we observe the sample-based transition-building subroutine outperform its more conservative constituents.

Transition-building subroutines. We analyzed the effects of each transition-building subroutine (AABB-, polytope-, and sample-based) across the case studies, reporting the model building and verification runtimes, *abstraction metrics*, and *verification metrics*. Abstractions were devised under fixed dimension sizes:

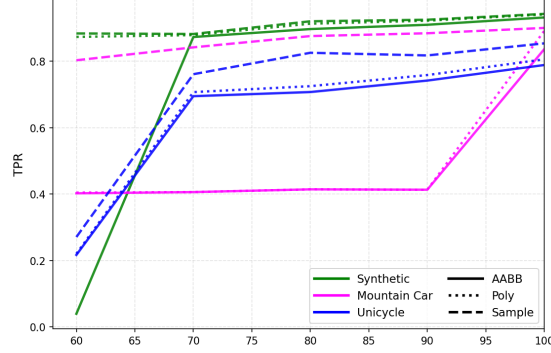


Fig. 3. TPR reported against model dimension size across all case studies and transition-building subroutines.

Table 1. Comparison of transition-building subroutines for each case study.

Case study	Method	Time (s)		Abstraction metrics					Verification
		Build	Verify	SLP	mSu	Su	$\overline{\text{Su}}$	Su_σ	TPR
Synthetic	AABB	0.138	0.235	0.003	4.07	1	9	1.44	0.931
	PT	1.028	0.157	0.002	3.64	1	5	1.12	0.941
	Sample	0.782	0.216	0.002	3.27	1	4	1.06	0.942
Mountain	AABB	0.179	0.158	0.037	4.64	1	80	3.60	0.836
	PT	2.514	0.261	0.033	4.37	1	51	2.82	0.891
	Sample	2.199	0.144	0.022	3.77	1	10	0.83	0.900
Unicycle	AABB	9.130	7.388	0.011	9.62	1	54	6.26	0.217
	PT	161.906	7.892	0.005	8.72	1	46	4.44	0.220
	Sample	43.430	8.238	0.001	6.68	1	22	1.66	0.271

for the synthetic case study, $|\hat{X}| = 100^2$; for mountain car, $|\hat{X}| = 100^2$; and for unicycle, $|\hat{X}| = 60^3$. We report these results under Tab. 1.

For the sample-based methods, we instantiate the desired certificate with confidence level $\delta = 0.01$ and significance threshold $\beta = 0.01$. Batches of 1000 samples were drawn uniformly from $\text{Unif}(X)$ until the missing mass upper bound $\overline{M}_N$ (Eq. 16 dropped below β , enabling us to assert the PAC guarantee in Eq. 17). For the synthetic case study, we needed 1439 batches. For mountain car, we required 1449 batches. Finally, for unicycle, we drew 19342 batches until the significance threshold was met. Therefore, the PAC guarantee asserted for each case study is $\Pr(M_N < 0.01) \geq 0.99$.

Our findings reveal that less conservative transition-construction methods produce sparser transition relations (fewer self-loops and fewer total transitions). We also observe a marked reduction in the successor-count variability (Su_σ) under sampling, especially for the nonlinear systems (mountain car and unicycle).

A likely explanation is that strong nonlinearities distort cell images away from rectangular shapes, causing AABB and polytope over-approximations to “wrap” over non-successor cells. This is not an issue in the sample-based construction because it does not admit any spurious transitions.

All together, these effects compound downstream into better verification outcomes for less conservative transition-building subroutines. We observe an increase in TPR when comparing the AABB- and polytope-based technique, and sample-based over polytope-based technique.

Table 2. Verification statistics before and after *reachable-set* self-loop erasure.

Case study	Method	Time (s)		Before		After	
		Build	Verify	SLP	TPR	SLP	TPR
Synthetic	AABB	0.162	0.404	0.003	0.931	0.000	0.931
	PT	1.075	0.324	0.003	0.941	0.000	0.941
	Sample	0.802	0.379	0.003	0.942	0.000	0.942
Mountain	AABB	0.256	0.320	0.037	0.836	0.004	0.895
	PT	2.534	0.420	0.033	0.891	0.004	0.898
	Sample	2.216	0.298	0.022	0.900	0.001	0.933
Unicycle	AABB	42.466	40.134	0.011	0.192	0.000	0.196
	PT	825.700	42.429	0.005	0.195	0.000	0.401
	Sample	243.865	46.440	0.001	0.240	0.000	0.630

Self-loop erasure. We examined the effects of assured self-loop erasure on the abstractions formed above (under the same dimension sizes). In particular, we report side-by-side the SLP, TPR, and SR before and after self-loop erasure. The results of reachable-set-based erasure are reported in Tab. 2, and the results for sample-based erasure are reported in Tab. 3.

For the sample-based technique, we set the desired confidence to $\gamma = 0.01$ and maximum trajectory length to $K = 50$. Sampling at least $N = 469$ trajectories yield the guarantee $\Pr(q_\tau \geq 0.99) \geq 0.99$. In words, with at least 99% confidence, at least 99% of K -length trajectories escape the cell. When observed that all N trajectories escape, this guarantee asserts high confidence that erasing a self loop from that particular state is acceptable.

Our findings reveal an obvious reduction in SLP post-erasure across all case studies and transition-building subroutines. For the conservative AABB-based erasure, Tab. 2 shows a nearly complete deletion of self loops in the synthetic and unicycle case study, and a significant reduction in SLP in the mountain care case study. In the sample-based regime, we observe in Tab. 3 complete deletion of self loops in the mountain car case study. The residual SLP in the synthetic case study is likely due to the equilibrium point in the concrete system (rending it impossible to certify self-loop erasure on the state that spans this equilibrium).

Table 3. Verification statistics before and after *sample-based* self-loop erasure. Confidence level is set to $\gamma = 0.01$; sample count is set to $N = 469$; this asserts the guarantee $\Pr(p_\tau \geq 0.99) \geq 0.99$ for each successfully certified erasure.

Case study	Method	Time (s)		Before		After	
		Build	Verify	SLP	TPR	SLP	TPR
Synthetic	AABB	0.162	0.404	0.003	0.931	0.000	0.931
	PT	1.075	0.324	0.003	0.941	0.000	0.941
	Sample	0.802	0.379	0.003	0.942	0.000	0.942
Mountain	AABB	0.225	0.364	0.037	0.836	0.000	0.895
	PT	2.536	0.467	0.033	0.891	0.000	0.898
	Sample	2.221	0.346	0.022	0.900	0.000	0.933
Unicycle	AABB	41.905	41.142	0.011	0.192	0.000	0.196
	PT	825.047	41.432	0.005	0.195	0.000	0.401
	Sample	242.999	47.437	0.001	0.240	0.000	0.630

These results transfer directly to downstream verification outcomes. In the mountain car case, we see marginal improvement in TPR post-erasure. In the unicycle case, we observe substantial improvement in the TPR. Finally, for the synthetic case, we do not observe any change likely because most (if not all) self loops lie near the equilibrium where concrete step sizes are small.

CEGAR. Finally, we assess the impact of CEGAR on verification outcomes across the case studies. Initial abstractions use a grid resolution of 100 per dimension for the synthetic system ($|\hat{X}| = 100^2$) and 20 per dimension for mountain car ($|\hat{X}| = 20^2$) and unicycle ($|\hat{X}| = 20^3$). We report out findings in Tab. 4.

CEGAR increases model size and construction time substantially: the refined abstractions contain roughly 11500 states (synthetic), 6500 states (mountain car), and 15000 states (unicycle). Nonetheless, CEGAR consistently reduces SLP, while leaving mSu largely unchanged (with only minor fluctuations).

In terms of verification performance, CEGAR yields a modest TPR improvement on the synthetic system and a pronounced improvement on mountain car (likely due to indirectly removing self-loop-driven counterexamples during refinement). In contrast, TPR is essentially unchanged for unicycle, which we attribute to its high SLP: the abstraction admits many trivial self-loop counterexamples that are difficult for CEGAR to eliminate without more targeted erasure.

10 Conclusion

Symbolic model checking is an effective approach to verify rich temporal properties of CPS, but its guarantees only transfer to the concrete system if its abstraction is conservative. In this tutorial we explored a rigorous workflow to build such abstractions while addressing common pitfalls encountered in model building. Specifically, the discussed workflow involved covering the state space with a

Table 4. Verification runtime statistics before and after CEGAR. Build time is reported in minutes, while verification time is reported in seconds.

Case study	Method	Time (m, s)		Before			After		
		Build	Verify	SLP	mSu	TPR	SLP	mSu	TPR
Synthetic	AABB	113.32	3.64	0.003	4.07	0.931	0.003	4.54	0.945
	PT	110.86	3.66	0.003	3.64	0.941	0.003	3.95	0.958
Mountain	AABB	76.90	1.66	0.657	4.26	0.081	0.197	5.25	0.430
	PT	148.55	1.75	0.643	4.15	0.081	0.173	4.62	0.422
Unicycle	AABB	33.86	0.33	0.280	8.90	0.094	0.155	10.10	0.045
	PT	89.26	0.27	0.278	8.35	0.094	0.148	6.23	0.093

state abstraction, synthesizing a transition relation that conservatively approximates the concrete dynamics, purging degenerate transitions, and soundly lifting the LTL specification from the concrete to the abstract space. We validated this workflow on three case studies, illustrating the effects of each transition-building method and refinement method on downstream verification performance. Looking forward, we will explore broader CPS case studies to stress-test this workflow and extensions of these techniques to stochastic CPS.

References

1. Alur, R.: Formal verification of hybrid systems. In: Proceedings of the ninth ACM international conference on Embedded software. pp. 273–278. ACM, Taipei Taiwan (Oct 2011). <https://doi.org/10.1145/2038642.2038685>, <https://dl.acm.org/doi/10.1145/2038642.2038685>
2. Alur, R., Dang, T., Ivančić, F.: Counterexample-guided predicate abstraction of hybrid systems. Theoretical Computer Science **354**(2), 250–271 (Mar 2006). <https://doi.org/10.1016/j.tcs.2005.11.026>, <https://www.sciencedirect.com/science/article/pii/S0304397505008662>
3. Alur, R., Dang, T., Ivančić, F.: Predicate abstraction for reachability analysis of hybrid systems. ACM Trans. Embed. Comput. Syst. **5**(1), 152–199 (Feb 2006). <https://doi.org/10.1145/1132357.1132363>, <https://dl.acm.org/doi/10.1145/1132357.1132363>
4. Baier, C., Katoen, J.P.: Principles of Model Checking. MIT Press, Cambridge, MA, USA (Apr 2008)
5. Berend, D., Kontorovich, A.: The missing mass problem. Statistics & Probability Letters **82**(6), 1102–1110 (Jun 2012). <https://doi.org/10.1016/j.spl.2012.02.014>, <https://www.sciencedirect.com/science/article/pii/S0167715212000612>
6. Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W.: OpenAI Gym (Jun 2016), <https://arxiv.org/abs/1606.01540v1>
7. Burch, J.R., Clarke, E.M., McMillan, K.L., Dill, D.L., Hwang, L.J.: Symbolic model checking: 10^{20} states and beyond. Information and Computation **98**(2), 142–170 (1992)

8. Casagrande, A.: `albertocasagrande/pyModelChecking` (Dec 2025), <https://github.com/albertocasagrande/pyModelChecking>, original-date: 2015-03-10T09:22:24Z
9. Chadha, R., Viswanathan, M.: A counterexample-guided abstraction-refinement framework for markov decision processes. *ACM Trans. Comput. Logic* **12**(1), 1:1–1:49 (Nov 2010). <https://doi.org/10.1145/1838552.1838553>, <https://dl.acm.org/doi/10.1145/1838552.1838553>
10. Chatterjee, K., Chmelík, M., Daca, P.: CEGAR for Qualitative Analysis of Probabilistic Systems. In: Biere, A., Bloem, R. (eds.) *Computer Aided Verification*. pp. 473–490. Springer International Publishing, Cham (2014). https://doi.org/10.1007/978-3-319-08867-9_31
11. Chen, X., Sankaranarayanan, S.: Reachability Analysis for Cyber-Physical Systems: Are We There Yet? In: Deshmukh, J.V., Havelund, K., Perez, I. (eds.) *NASA Formal Methods*. pp. 109–130. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-06773-0_6
12. Clarke, E., Fehnker, A., Han, Z., Krogh, B., Ouaknine, J., Stursberg, O., Theobald, M.: Abstraction and Counterexample-Guided Refinement in Model Checking of Hybrid Systems. *International Journal of Foundations of Computer Science* **14**(04), 583–604 (Aug 2003). <https://doi.org/10.1142/S012905410300190X>, <https://www.worldscientific.com/doi/abs/10.1142/S012905410300190X>, publisher: World Scientific Publishing Co.
13. Clarke, E., Fehnker, A., Han, Z., Krogh, B., Stursberg, O., Theobald, M.: Verification of Hybrid Systems Based on Counterexample-Guided Abstraction Refinement. In: Goos, G., Hartmanis, J., Van Leeuwen, J., Garavel, H., Hatcliff, J. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*, vol. 2619, pp. 192–207. Springer Berlin Heidelberg, Berlin, Heidelberg (2003). https://doi.org/10.1007/3-540-36577-X_14, http://link.springer.com/10.1007/3-540-36577-X_14, series Title: Lecture Notes in Computer Science
14. Clarke, E., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-Guided Abstraction Refinement. In: Emerson, E.A., Sistla, A.P. (eds.) *Computer Aided Verification*. pp. 154–169. Springer, Berlin, Heidelberg (2000). https://doi.org/10.1007/10722167_15
15. Clarke, E.M., Emerson, E.A., Sistla, A.P.: Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Transactions on Programming Languages and Systems* **8**(2), 244–263 (1986). <https://doi.org/10.1145/5397.5399>
16. Dierks, H., Kupferschmid, S., Larsen, K.G.: Automatic Abstraction Refinement for Timed Automata. In: Raskin, J.F., Thiagarajan, P.S. (eds.) *Formal Modeling and Analysis of Timed Systems*. pp. 114–129. Springer, Berlin, Heidelberg (2007). https://doi.org/10.1007/978-3-540-75454-1_10
17. Doyen, L., Frehse, G., Pappas, G.J., Platzer, A.: Verification of Hybrid Systems. In: Clarke, E.M., Henzinger, T.A., Veith, H., Bloem, R. (eds.) *Handbook of Model Checking*, pp. 1047–1110. Springer International Publishing, Cham (2018). https://doi.org/10.1007/978-3-319-10575-8_30, http://link.springer.com/10.1007/978-3-319-10575-8_30
18. Dubins, L.E.: On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents. *American Journal of Mathematics* **79**(3), 497–516 (1957). <https://doi.org/10.2307/2372560>, <https://www.jstor.org/stable/2372560>, publisher: The Johns Hopkins University Press

19. Fehnker, A., Clarke, E., Jha, S.K., Krogh, B.: Refining Abstractions of Hybrid Systems Using Counterexample Fragments. In: Morari, M., Thiele, L. (eds.) *Hybrid Systems: Computation and Control*. pp. 242–257. Springer, Berlin, Heidelberg (2005). https://doi.org/10.1007/978-3-540-31954-2_16
20. Frehse, G.: PHAVer: algorithmic verification of hybrid systems past HyTech. *International Journal on Software Tools for Technology Transfer* **10**(3), 263–279 (Jun 2008). <https://doi.org/10.1007/s10009-007-0062-x>, <http://link.springer.com/10.1007/s10009-007-0062-x>
21. Frehse, G., Le Guernic, C., Donzé, A., Cotton, S., Ray, R., Lebeltel, O., Ripado, R., Girard, A., Dang, T., Maler, O.: SpaceEx: Scalable Verification of Hybrid Systems. In: Gopalakrishnan, G., Qadeer, S. (eds.) *Computer Aided Verification*. pp. 379–395. Springer, Berlin, Heidelberg (2011). https://doi.org/10.1007/978-3-642-22110-1_30
22. Good, I.: Turing’s anticipation of empirical bayes in connection with the crypt-analysis of the naval enigma *. *Journal of Statistical Computation and Simulation* **66**(2), 101–111 (May 2000). <https://doi.org/10.1080/00949650008812016>, <https://doi.org/10.1080/00949650008812016>, publisher: Taylor & Francis eprint: <https://doi.org/10.1080/00949650008812016>
23. Guéguen, H., Lefebvre, M.A., Zaytoon, J., Nasri, O.: Safety verification and reachability analysis for hybrid systems. *Annual Reviews in Control* **33**(1), 25–36 (Apr 2009). <https://doi.org/10.1016/j.arcontrol.2009.03.002>, <https://www.sciencedirect.com/science/article/pii/S1367578809000078>
24. Henzinger, T.: The theory of hybrid automata. In: *Proceedings 11th Annual IEEE Symposium on Logic in Computer Science*. pp. 278–292 (Jul 1996). <https://doi.org/10.1109/LICS.1996.561342>, <https://ieeexplore.ieee.org/document/561342>, ISSN: 1043-6871
25. Henzinger, T.A., Jhala, R., Majumdar, R., Sutre, G.: Lazy abstraction. In: *Proceedings of the 29th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. pp. 58–70. POPL ’02, Association for Computing Machinery, New York, NY, USA (Jan 2002). <https://doi.org/10.1145/503272.503279>, <https://dl.acm.org/doi/10.1145/503272.503279>
26. Hermanns, H., Wachter, B., Zhang, L.: Probabilistic CEGAR. In: Gupta, A., Malik, S. (eds.) *Computer Aided Verification*. pp. 162–175. Springer, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-70545-1_16
27. Kloetzer, M., Belta, C.: A Fully Automated Framework for Control of Linear Systems from Temporal Logic Specifications. *IEEE Transactions on Automatic Control* **53**(1), 287–297 (Feb 2008). <https://doi.org/10.1109/TAC.2007.914952>, <http://ieeexplore.ieee.org/document/4459804/>
28. McAllester, D.A., Schapire, R.E.: On the Convergence Rate of Good-Turing Estimators. In: *Proceedings of the Thirteenth Annual Conference on Computational Learning Theory*. pp. 1–6. COLT ’00, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (Jun 2000)
29. Sloth, C., Wisniewski, R.: Algorithmic Approach to Abstracting Linear Systems by Timed Automata*. *IFAC Proceedings Volumes* **44**(1), 4546–4551 (Jan 2011). <https://doi.org/10.3182/20110828-6-IT-1002.02568>, <https://linkinghub.elsevier.com/retrieve/pii/S1474667016443247>

A Appendix

Unicycle state controller. The unicycle controller used in our experiments is a smooth, deterministic state-feedback law that selects the angular velocity u by steering the system toward a desired heading $\theta_d(x)$ computed from the current state. Recall the goal region centered at $y^{\text{goal}} = (y_1^{\text{goal}}, y_2^{\text{goal}})$ with radius r^{goal} , and the obstacle centered at $y^{\text{obs}} = (y_1^{\text{obs}}, y_2^{\text{obs}})$ with radius r^{obs} .

At a high level, the controller forms a planar guidance vector by combining an *attractive* component that points toward the goal with a *repulsive* component that pushes the system away from the obstacle. The resulting vector determines the desired heading.

Let $y = (y_1, y_2)$ denote the positional component of the state x , and define the displacement from the obstacle center by

$$d^{\text{obs}}(x) = y - y^{\text{obs}}.$$

Its Euclidean norm $\|d^{\text{obs}}(x)\|$ is the distance from the system to the obstacle center. We then define the obstacle clearance as the distance to the obstacle boundary,

$$c(x) = \|d^{\text{obs}}(x)\| - r^{\text{obs}},$$

and map this clearance through an exponential weighting,

$$\tilde{c}(x) = e^{-\alpha c(x)},$$

where $\alpha \in \mathbb{R}_{>0}$ controls the rate of decay. Thus, when the system lies on the obstacle boundary ($c(x) = 0$), we have $\tilde{c}(x) = 1$, and this quantity decays toward 0 as the system moves farther from the obstacle.

Using this weight, the repulsive guidance vector is defined as

$$v^{\text{rep}}(x) = \left[\frac{\tilde{c}(x)}{\|d^{\text{obs}}(x)\|^3 + \epsilon} \right] d^{\text{obs}}(x),$$

where $\epsilon \in \mathbb{R}_{>0}$ is a small constant included to avoid singular behavior near the obstacle center. The attractive component is simply

$$v^{\text{att}}(x) = y^{\text{goal}} - y.$$

These two terms are combined as

$$v(x) = k^{\text{rep}} v^{\text{rep}}(x) + k^{\text{att}} v^{\text{att}}(x),$$

where k^{rep} and k^{att} are tunable repulsion and attraction gains, respectively. Intuitively, when the system is far from the obstacle, $v^{\text{rep}}(x)$ is small and the heading is dominated by attraction toward the goal; near the obstacle, the repulsive term grows in influence and steers the system away from collision.

Finally, the desired heading is extracted from the planar vector $v(x)$ via

$$\theta_d(x) = \arctan(v_1(x), v_2(x)), \quad v = (v_1, v_2)$$

and the control input is chosen as

$$u(x) = u_{\max} \tanh(k_{\theta}(\theta_d(x) - \theta)),$$

where $u_{\max}$ sets the allowable turn rate (so that $u \in [-u_{\max}, u_{\max}]$) and k_{θ} is a tunable gain on the heading error.

Concrete categorical probabilities. Fix an abstract state $\hat{x} \in \hat{X}$ and consider its concretization $\psi^{-1}(\hat{x}) \subseteq X$. Define the one-step pre-image of this concretization under f as $\text{Pre}(\psi^{-1}(\hat{x})) = \{x \in X : \psi(f(x)) = \hat{x}\}$. Intuitively, $\text{Pre}(\psi^{-1}(\hat{x}))$ collects exactly those concrete states that land in $\psi^{-1}(\hat{x})$ after one time step.

We next partition the state space into regions that correspond to specific abstract transitions. Concretely, for each pair $(\hat{x}, \hat{x}') \in \hat{X} \times \hat{X}$, define the region of states that start in $\psi^{-1}(\hat{x})$ and map in one step into $\psi^{-1}(\hat{x}')$:

$$\mathcal{R} = \{\text{Pre}(\psi^{-1}(\hat{x}')) \cap \psi^{-1}(\hat{x}) \mid \forall \hat{x}', \hat{x} \in \hat{X}\}. \quad (30)$$

Denote an element of this collection by $R_{\ell} \in \mathcal{R}$, where $\ell \in \{1, \dots, |\mathcal{R}|\}$. By construction, any sample $x \in R_{\ell}$ maps to the subset $\psi^{-1}(\hat{x}')$, and therefore witnesses the abstract transition from $\hat{x}$ to $\hat{x}'$. That is, $\psi(f(x)) = \hat{x}'$ for any $x \in R_{\ell}$, which forms the edge $(\hat{x}, \sigma, \hat{x}')$ in the sample-built transition relation F^* .

Assume that samples are drawn uniformly over X , i.e., $x \sim \text{Unif}(X)$. This induces a categorical distribution $\text{Cat}(F^*)$ over transition relations $(\psi(x), \sigma, \psi(f(x)))$ in F^* with categories $\ell = \{1, \dots, |F^*|\}$. Each triple $(\psi(x), \sigma, \psi(f(x)))$ with index ℓ has a probability mass q_{ℓ} in this $\text{Cat}(F^*)$. Intuitively, q_{ℓ} is the chance that after a single draw $x \sim \text{Unif}(X)$, the algorithm allocates the relation $(\psi(x), \sigma, \psi(f(x)))$ to F .

Concretely, the probability q_{ℓ} mass of category ℓ is precisely:

$$q_{\ell} = \frac{\lambda(R_{\ell})}{\lambda(X)}, \quad (31)$$

where $\lambda(\cdot)$ denotes Lebesgue measure (hypervolume).